

Data Cleaning AI Agent

A rule-driven agent that perceives data-quality issues in a dataset, decides on an appropriate cleaning action for each issue, applies it, and produces a full audit trail explaining every change it made.

1. What the agent does

The agent follows a perceive → decide → act → report loop, the same loop structure used by autonomous agents generally:

PERCEIVE — Profiles every column: dtype, missing values, duplicate rows, outliers, inconsistent casing/whitespace, values stored as the wrong type.

DECIDE — A rule engine selects an action per issue (impute vs. drop, cast vs. leave alone, clip vs. keep an outlier), based on how severe and how safe-to-fix the issue is.

ACT — The chosen action is applied to a working copy of the DataFrame.

REPORT — Every action, its rationale, and the number of rows it touched is logged, so the cleaning is auditable rather than a black box.

Cleaning rules implemented

- Standardizes column names (lowercase, underscores, trimmed).
- Strips leading/trailing whitespace from all text fields.
- Converts blank / whitespace-only cells into proper null values.
- Detects numbers stored as text (e.g. "\$1,200") and casts them to numeric.
- Detects dates stored as text in mixed formats and casts them to datetime.
- Handles missing values per-column: median for numeric; mode for text that is genuinely low-cardinality/categorical; leaves dates and high-cardinality free text (names, notes) as null rather than fabricating a value; and drops columns that are over 60% empty.
- Normalizes inconsistent capitalization in low-cardinality text columns (e.g. "usa" / "USA" / "Usa" → "Usa").
- Flags statistical outliers ($|z\text{-score}| > 3$) and clips them to the 1st–99th percentile range instead of deleting the row.
- Detects an ID-like column automatically and excludes it from duplicate detection, then drops true duplicate rows.

The `DataCleaningAgent` class also accepts an optional `llm_hook` callback, so an LLM (e.g. the Claude API) can be plugged in later to resolve ambiguous free-text values in natural language — the deterministic rule engine below runs fully offline and needs no API key.

2. Full Source Code

data_cleaning_agent.py

```
"""
Data Cleaning AI Agent
=====

A rule-driven "agent" that inspects a raw CSV/DataFrame, decides which
cleaning actions to apply column-by-column, executes them, and produces
a human-readable report of every decision it made (so the cleaning is
auditable, not a black box).

Why call it an "agent"?
-----
It follows the classic perceive -> decide -> act -> report loop:
1. PERCEIVE : profile each column (dtype, nulls, duplicates, outliers,
              inconsistent text casing/whitespace, mixed types, etc.)
2. DECIDE   : a rule engine picks an appropriate action per issue
              (impute / drop / standardize / clip / dedupe)
3. ACT      : the action is applied to the DataFrame
4. REPORT   : every action + rationale is logged into a structured
              cleaning report (JSON + text) so a human can audit it.

No external LLM call is required for this reference implementation
(it works fully offline / deterministically), but the same Agent class
can be pointed at an LLM (see `llm_hook`) to have a model decide edge
cases in natural language -- the hook is included below.

Usage
-----
python -m data_cleaning_agent.agent input.csv cleaned_output.csv report.json
# or, after `pip install -e .`:
clean-data input.csv cleaned_output.csv report.json

Author: Claude (Anthropic) - generated example
"""

from __future__ import annotations

import json
import re
import sys
from dataclasses import dataclass, field
from typing import Any, Callable, Optional

import numpy as np
import pandas as pd

# -----
# 1. Data structures for tracking what the agent did
# -----

@dataclass
class CleaningAction:
    column: str
    issue: str
    action: str
    details: str
    rows_affected: int = 0

@dataclass
class CleaningReport:
    actions: list[CleaningAction] = field(default_factory=list)
    rows_before: int = 0
    rows_after: int = 0
    cols_before: int = 0
    cols_after: int = 0

    def log(self, column: str, issue: str, action: str, details: str, rows_affected: int = 0):
        self.actions.append(CleaningAction(column, issue, action, details, rows_affected))

    def to_dict(self) -> dict:
```

```

        return {
            "summary": {
                "rows_before": self.rows_before,
                "rows_after": self.rows_after,
                "cols_before": self.cols_before,
                "cols_after": self.cols_after,
                "total_actions": len(self.actions),
            },
            "actions": [a.__dict__ for a in self.actions],
        }

    def to_text(self) -> str:
        lines = [
            "DATA CLEANING REPORT",
            "-" * 60,
            f"Rows: {self.rows_before} -> {self.rows_after}",
            f"Columns: {self.cols_before} -> {self.cols_after}",
            f"Total actions taken: {len(self.actions)}",
            "-" * 60,
        ]
        for a in self.actions:
            lines.append(
                f"[{a.column}] {a.issue} -> {a.action} "
                f"({a.rows_affected} rows) :: {a.details}"
            )
        return "\n".join(lines)

# -----
# 2. The Agent
# -----

class DataCleaningAgent:
    """
    A configurable agent that perceives data quality issues in a DataFrame
    and decides/acts on cleaning steps, producing a full audit trail.

    Parameters
    -----
    outlier_z_thresh : float
        Z-score beyond which a numeric value is flagged as an outlier.
    dup_subset : list[str] | None
        Columns to consider when detecting duplicate rows. None = all columns.
    llm_hook : Callable[[str], str] | None
        Optional callback the agent can use to ask an LLM to classify an
        ambiguous free-text value (e.g. normalizing "USA", "U.S.A", "United
        States" -> a canonical label). Not required to run the agent.
    """

    def __init__(
        self,
        outlier_z_thresh: float = 3.0,
        dup_subset: Optional[list[str]] = None,
        llm_hook: Optional[Callable[[str], str]] = None,
    ):
        self.outlier_z_thresh = outlier_z_thresh
        self.dup_subset = dup_subset
        self.llm_hook = llm_hook
        self.report = CleaningReport()

    # ---- PERCEIVE + DECIDE + ACT, orchestrated -----

    def clean(self, df: pd.DataFrame) -> pd.DataFrame:
        df = df.copy()
        self.report = CleaningReport()
        self.report.rows_before, self.report.cols_before = df.shape

        # Order matters: normalize representation first (names, whitespace,
        # dtypes, text casing) so that two rows which are "really" the same
        # record are recognized as duplicates. Deduplicate on that raw,
        # un-imputed data *before* filling in missing values -- otherwise an
        # imputed value (e.g. two rows both getting the same median) can look
        # like an accidental duplicate of an unrelated row and get dropped.
        df = self._standardize_column_names(df)
        df = self._strip_whitespace(df)

```

```

df = self._fix_dtypes(df)
df = self._normalize_text_columns(df)
df = self._drop_duplicate_rows(df)
df = self._handle_missing_values(df)
df = self._handle_outliers(df)

self.report.rows_after, self.report.cols_after = df.shape
return df

# ---- individual perception/decision/action steps -----

def _standardize_column_names(self, df: pd.DataFrame) -> pd.DataFrame:
    original = list(df.columns)
    new_cols = (
        df.columns.str.strip()
        .str.lower()
        .str.replace(r"^\w+", "_", regex=True)
        .str.strip("_")
    )
    if list(new_cols) != original:
        df.columns = new_cols
        self.report.log(
            "ALL", "inconsistent column naming", "standardize_column_names",
            f"{original} -> {list(new_cols)}",
        )
    return df

def _strip_whitespace(self, df: pd.DataFrame) -> pd.DataFrame:
    for col in df.select_dtypes(include=["object", "str"]).columns:
        before = df[col].copy()
        df[col] = df[col].apply(lambda x: x.strip() if isinstance(x, str) else x)
        changed = (before != df[col]) & before.notna()
        if changed.sum() > 0:
            self.report.log(
                col, "leading/trailing whitespace", "strip",
                "trimmed whitespace from text values", int(changed.sum()),
            )
        # Whitespace-only cells become "" after stripping -- treat as missing
        blank_mask = df[col] == ""
        n_blank = int(blank_mask.sum())
        if n_blank > 0:
            df.loc[blank_mask, col] = np.nan
            self.report.log(
                col, "blank/whitespace-only cells", "convert_blank_to_null",
                "empty strings after stripping treated as missing values",
                n_blank,
            )
    return df

def _fix_dtypes(self, df: pd.DataFrame) -> pd.DataFrame:
    for col in df.columns:
        if not (df[col].dtype == object or str(df[col].dtype) == "str"):
            continue

        # try numeric coercion (handles "$1,200", "1200 ", "3.5%")
        cleaned = (
            df[col]
            .astype(str)
            .str.replace(r"[$,%]", "", regex=True)
            .str.strip()
        )
        numeric = pd.to_numeric(cleaned, errors="coerce")
        non_null = df[col].notna()
        success_rate = numeric[non_null].notna().mean() if non_null.sum() else 0

        if success_rate >= 0.8:
            n_changed = int((df[col].notna() & numeric.isna()).sum())
            df[col] = numeric
            self.report.log(
                col, "numeric stored as text", "cast_to_numeric",
                f"parsed currency/percent symbols; {n_changed} values "
                "could not be parsed and became NaN",
            )
            continue
        # try datetime coercion

```

```

dt = pd.to_datetime(df[col], errors="coerce", format="mixed")
non_null = df[col].notna()
success_rate = dt[non_null].notna().mean() if non_null.sum() else 0
if success_rate >= 0.8:
    df[col] = dt
    self.report.log(
        col, "date stored as text", "cast_to_datetime",
        "parsed inconsistent date formats into datetime64",
    )
return df

def _handle_missing_values(self, df: pd.DataFrame) -> pd.DataFrame:
    for col in df.columns:
        n_missing = df[col].isna().sum()
        if n_missing == 0:
            continue
        pct_missing = n_missing / len(df)

        if pct_missing > 0.6:
            df = df.drop(columns=[col])
            self.report.log(
                col, f"{pct_missing:.0%} missing", "drop_column",
                "too sparse to impute reliably", int(n_missing),
            )
            continue

        if pd.api.types.is_numeric_dtype(df[col]):
            fill_value = df[col].median()
            df[col] = df[col].fillna(fill_value)
            self.report.log(
                col, f"{n_missing} missing numeric values", "impute_median",
                f"filled with column median ({fill_value:.3g})", int(n_missing),
            )
        elif pd.api.types.is_datetime64_any_dtype(df[col]):
            df = df # leave date gaps as-is; imputing dates is unsafe
            self.report.log(
                col, f"{n_missing} missing dates", "leave_as_null",
                "imputing dates would fabricate events; left as NaT",
                int(n_missing),
            )
        else:
            # Mode-imputation only makes sense for genuinely categorical
            # columns (a handful of repeated values). For high-cardinality
            # free text (notes, comments, names) picking "the most common
            # value" and stamping it onto unrelated rows would fabricate
            # misleading content, so those are left as null instead.
            non_null = df[col].dropna()
            n_unique = non_null.nunique()
            # A column only "looks categorical" if (a) it has a capped
            # number of distinct values, and (b) at least one value
            # repeats -- i.e. it isn't just a column where every row
            # happens to be distinct (names, free-text notes, IDs).
            # Requiring repetition rather than a fixed ratio holds up
            # on both small and large samples.
            has_repeats = n_unique < len(non_null)
            looks_categorical = n_unique <= 20 and has_repeats
            if looks_categorical:
                mode = df[col].mode(dropna=True)
                fill_value = mode.iloc[0] if not mode.empty else "Unknown"
                df[col] = df[col].fillna(fill_value)
                self.report.log(
                    col, f"{n_missing} missing categorical values",
                    "impute_mode",
                    f"filled with most frequent value ('{fill_value}']",
                    int(n_missing),
                )
            else:
                self.report.log(
                    col, f"{n_missing} missing free-text values",
                    "leave_as_null",
                    "high-cardinality free text; imputing a 'most '
                    'common' value would fabricate misleading content",
                    int(n_missing),
                )
    return df

```

```

def _normalize_text_columns(self, df: pd.DataFrame) -> pd.DataFrame:
    for col in df.select_dtypes(include=["object", "str"]).columns:
        if df[col].nunique(dropna=True) == 0:
            continue
        # Detect columns that look categorical (low cardinality) and
        # normalize casing so "usa", "USA", "Usa" collapse to one label.
        n_unique = df[col].nunique(dropna=True)
        if n_unique <= max(20, int(0.1 * len(df))):
            before_unique = df[col].dropna().unique().tolist()
            normalized = df[col].apply(
                lambda x: x.title() if isinstance(x, str) else x
            )
            collapsed = normalized.dropna().unique().tolist()
            if len(collapsed) < len(before_unique):
                df[col] = normalized
                self.report.log(
                    col, "inconsistent capitalization", "normalize_case",
                    f"{len(before_unique)} raw variants -> {len(collapsed)} "
                    "canonical values via title-casing",
                )
    return df

def _handle_outliers(self, df: pd.DataFrame) -> pd.DataFrame:
    for col in df.select_dtypes(include=[np.number]).columns:
        series = df[col]
        std = series.std()
        if not std or np.isnan(std):
            continue
        z = (series - series.mean()) / std
        mask = z.abs() > self.outlier_z_thresh
        n_outliers = int(mask.sum())
        if n_outliers == 0:
            continue
        lower, upper = series.quantile(0.01), series.quantile(0.99)
        df.loc[mask, col] = series[mask].clip(lower, upper).round(2)
        self.report.log(
            col, f"{n_outliers} statistical outliers (|z| > {self.outlier_z_thresh})",
            "clip_to_1st_99th_percentile",
            f"clipped extreme values to [{lower:.3g}, {upper:.3g}]",
            n_outliers,
        )
    return df

def _drop_duplicate_rows(self, df: pd.DataFrame) -> pd.DataFrame:
    subset = self.dup_subset
    if subset is None:
        # Auto-detect ID-like columns (unique per row, name suggests an
        # identifier) and exclude them -- two rows that are identical in
        # every *content* column are still duplicates even if a
        # surrogate ID column differs.
        id_like = [
            c for c in df.columns
            if re.search(r"^(|_)id($|_)", c) and df[c].is_unique
        ]
        subset = [c for c in df.columns if c not in id_like] or None
    if id_like:
        self.report.log(
            "ALL", "id-like column(s) detected", "exclude_from_dup_check",
            f"excluded {id_like} from duplicate comparison since a "
            "surrogate id will always be unique",
        )

    dup_mask = df.duplicated(subset=subset, keep="first")
    n_dupes = int(dup_mask.sum())
    if n_dupes > 0:
        df = df[~dup_mask].reset_index(drop=True)
        self.report.log(
            "ALL", f"{n_dupes} duplicate rows", "drop_duplicates",
            f"kept first occurrence, subset={subset or 'all columns'}",
            n_dupes,
        )
    return df

```

```

# 3. CLI entry point
# -----

def main():
    if len(sys.argv) < 4:
        print("Usage: python data_cleaning_agent.py <input.csv> <output.csv> <report.json>")
        sys.exit(1)

    input_path, output_path, report_path = sys.argv[1:4]

    df = pd.read_csv(input_path)
    agent = DataCleaningAgent()
    cleaned = agent.clean(df)

    cleaned.to_csv(output_path, index=False)
    with open(report_path, "w") as f:
        json.dump(agent.report.to_dict(), f, indent=2, default=str)

    print(agent.report.to_text())
    print(f"\nCleaned data written to: {output_path}")
    print(f"Report written to:      {report_path}")

if __name__ == "__main__":
    main()

```

3. Sample Input

A deliberately messy 12-row customer CSV: inconsistent column names, stray whitespace, currency symbols, mixed date formats, inconsistent country capitalization, missing values, an outlier purchase amount, and one duplicate record.

Customer ID	Full Name	Country	Signup Date	Purchase Amount	Notes
1	Alice Smith	usa	2023-01-05	\$1200	Regular customer
2	bob jones	USA	01/06/2023	950	
3	Alice Smith	usa	2023-01-05	\$1200	Regular customer
4	Carlos Diaz	Mexico	2023/01/07	1100.50	VIP
5	Dana Lee	CANADA	2023-01-08		New signup
6	Evan Wright	canada	2023-01-09	87000	Bulk order - unusually large
7	Farah Khan		2023-01-10	875	
8	George Owusu	Nigeria	2023-01-11	990	
9	Hana Kim	usa	not_a_date	1050	
10	Ivan Petrov	Russia	2023-01-13	1020	
11		USA	2023-01-14	940	missing name
12	Julia Roberts	Usa	2023-01-15	30	possible data entry error

4. Sample Output (Cleaned)

Result of running: `python data_cleaning_agent.py sample_input.csv cleaned_output.csv report.json`

customer_id	full_name	country	signup_date	purchase_amount	notes
1	Alice Smith	Usa	2023-01-05	1200.0	Regular customer
2	bob jones	Usa	2023-01-06	950.0	
4	Carlos Diaz	Mexico	2023-01-07	1100.5	VIP
5	Dana Lee	Canada	2023-01-08	1005.0	New signup
6	Evan Wright	Canada	2023-01-09	78420.0	Bulk order - unusually large
7	Farah Khan	Usa	2023-01-10	875.0	
8	George Owusu	Nigeria	2023-01-11	990.0	
9	Hana Kim	Usa		1050.0	
10	Ivan Petrov	Russia	2023-01-13	1020.0	
11		Usa	2023-01-14	940.0	missing name
12	Julia Roberts	Usa	2023-01-15	30.0	possible data entry error

5. Agent Cleaning Report (Audit Log)

This is the agent's own explanation of every decision it made, generated automatically alongside the cleaned file (also saved as `report.json` for programmatic use):

```
DATA CLEANING REPORT
=====
Rows:      12 -> 11
Columns:   6 -> 6
Total actions taken: 21
-----
[ALL] inconsistent column naming -> standardize_column_names (0 rows) :: ['Customer ID ', ' Full
      Name', ' Country ', ' Signup Date', ' Purchase Amount', ' Notes'] -> ['customer_id',
      'full_name', 'country', 'signup_date', 'purchase_amount', 'notes']
[full_name] leading/trailing whitespace -> strip (12 rows) :: trimmed whitespace from text values
[full_name] blank/whitespace-only cells -> convert_blank_to_null (1 rows) :: empty strings after
      stripping treated as missing values
[country] leading/trailing whitespace -> strip (12 rows) :: trimmed whitespace from text values
[country] blank/whitespace-only cells -> convert_blank_to_null (1 rows) :: empty strings after
      stripping treated as missing values
[signup_date] leading/trailing whitespace -> strip (12 rows) :: trimmed whitespace from text values
[purchase_amount] leading/trailing whitespace -> strip (12 rows) :: trimmed whitespace from text
      values
[purchase_amount] blank/whitespace-only cells -> convert_blank_to_null (1 rows) :: empty strings
      after stripping treated as missing values
[notes] leading/trailing whitespace -> strip (12 rows) :: trimmed whitespace from text values
[notes] blank/whitespace-only cells -> convert_blank_to_null (5 rows) :: empty strings after
      stripping treated as missing values
[signup_date] date stored as text -> cast_to_datetime (0 rows) :: parsed inconsistent date formats
      into datetime64
[purchase_amount] numeric stored as text -> cast_to_numeric (0 rows) :: parsed currency/percent
      symbols; 0 values could not be parsed and became NaN
[country] inconsistent capitalization -> normalize_case (0 rows) :: 8 raw variants -> 5 canonical
      values via title-casing
[ALL] id-like column(s) detected -> exclude_from_dup_check (0 rows) :: excluded ['customer_id'] from
      duplicate comparison since a surrogate id will always be unique
[ALL] 1 duplicate rows -> drop_duplicates (1 rows) :: kept first occurrence, subset=['full_name',
      'country', 'signup_date', 'purchase_amount', 'notes']
[full_name] 1 missing free-text values -> leave_as_null (1 rows) :: high-cardinality free text;
      imputing a 'most common' value would fabricate misleading content
[country] 1 missing categorical values -> impute_mode (1 rows) :: filled with most frequent value
      ('Usa')
[signup_date] 1 missing dates -> leave_as_null (1 rows) :: imputing dates would fabricate events;
      left as NaT
[purchase_amount] 1 missing numeric values -> impute_median (1 rows) :: filled with column median
      (1e+03)
[notes] 5 missing free-text values -> leave_as_null (5 rows) :: high-cardinality free text; imputing
      a 'most common' value would fabricate misleading content
[purchase_amount] 1 statistical outliers (|z| > 3.0) -> clip_to_1st_99th_percentile (1 rows) ::
      clipped extreme values to [114, 7.84e+04]

Cleaned data written to: cleaned_output.csv
Report written to:      report.json
```

6. How to Run It Yourself

```
pip install pandas numpy
python data_cleaning_agent.py sample_input.csv cleaned_output.csv report.json
```

Swap in any CSV as the first argument; the agent will profile it, clean it, and write both the cleaned CSV and the JSON audit report.